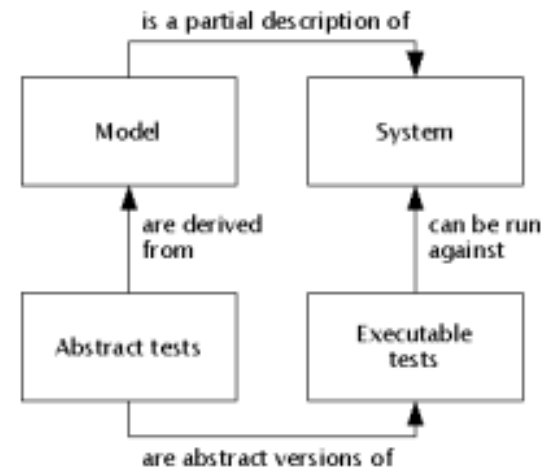




Model-based Testing

Model-based testing

- Use of a **model** of the **System Under Test (SUT)** for
 - generating and
 - executing test cases



Model-Based testing

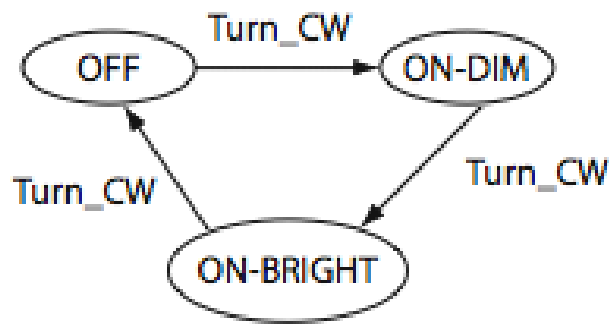
- The Model can be seen as abstract specification of the SUT
- The model is expressed in a formal language.
- The tests generated from the model aim to guarantee that the SUT is a correct implementation of the model.



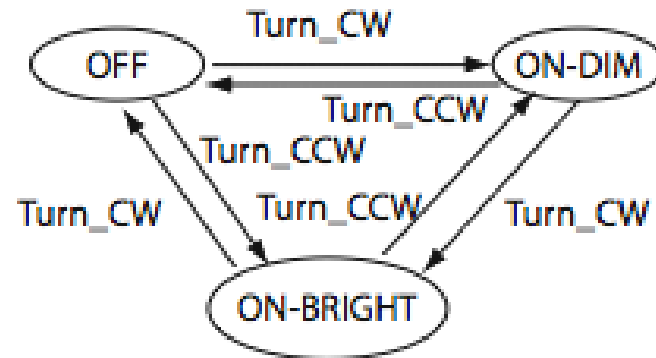
Finite state machines

FSM: lamp example

Simple three state lamp behavior:



(a)



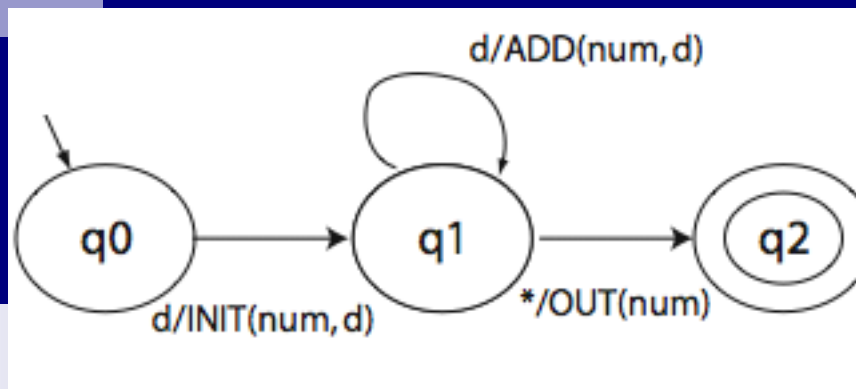
(b)

(a) Lamp switch can be turned clockwise.

(b) Lamp switch can be turned clockwise and counterclockwise..

FSM: Actions with state transitions

Machine to convert a sequence of decimal digits to an integer:



(a) Notice ADD, INIT, ADD, OUT actions.

(b) INIT: Initialize num. ADD: Add to num. OUT: Output num.

Finite Automaton

- A Finite Automaton consists of:
 - a set of states Q
 - a set of inputs Σ
 - an initial state q_0
 - a set of final states T
 - Accepting states
 - a function $F: Q \times \Sigma \rightarrow Q$ called the transition (next-state) function.
- Can be represented as state diagrams in which transitions are labeled with inputs.

Finite automaton

- A FA **accepts** a sequence of inputs if starting from the initial state q_0
 - for each input σ in the sequence there exist a transition (q, σ) , leading to some state q' , and
 - at the end of the sequence the automaton is found at a terminal state of set T .
- The set of sequences an automaton accepts is called the **language** of the automaton.

Transducers

- Transducers are finite automata which also generate outputs.
 - The machine has an output alphabet Γ
 - And an output function
 - $\omega: Q \times \Sigma \rightarrow \Gamma$ (Mealy machine)
- Transitions are labeled with input/output pairs

FSM (Mealy machine): Formal definition

An FSM (Mealy) is a 6-tuple: $(X, Y, Q, q_0, \delta, O)$, where:

X is a finite set of input symbols also known as the **input alphabet**.

Y is a finite set of output symbols also known as the **output alphabet**,

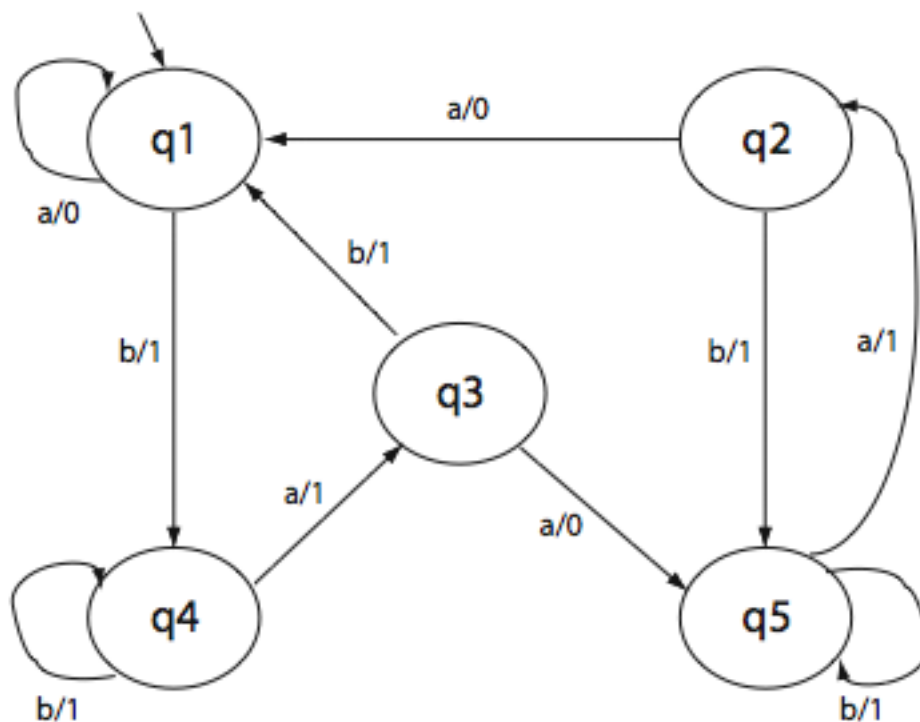
Q is a finite set **states**,

q_0 in Q is the **initial state**,

$\delta: Q \times X \rightarrow Q$ is a next-state or **state transition function**, and

$O: Q \times X \rightarrow Y$ is an **output function**

Example



Requirements specification or design specification?

An FSM could serve any of two roles:

- as a **specification** of the required behavior and/or
- as a **design** artifact according to which an application is to be implemented.

The role assigned to an FSM depends on whether it is a part of the requirements specification or of the design specification.

Note that FSMs are a part of UML design notation.

Where are FSMs used?

Modeling GUIs, network protocols, pacemakers, Teller machines, WEB applications, safety software modeling in nuclear plants, and many more.

While the FSM's considered in examples are abstract machines, they are abstractions of many real-life systems.

FSMs can be used to abstract the control structure of any program.



Stream X-machines

Extended FSM

In many instances in order to represent more clearly the control structure of a computer program a more complex state-based formalism is needed.

Extended FSM

An FSM always performs a transition on a given input from one state to another an

EFSM however only performs a transition when a given set of conditions have been satisfied.

Extended FSM

- An EFSM allows you to also use variables for example you could implement a for loop.
- This allows you to easily model algorithms that would otherwise require a combinatorial explosion of states.
- A Stream X-machine is a type of EFSM.

Stream X-Machines

- A Stream X-Machine (SXM) is like a FA with the following differences:
 - The machine has **memory**
 - transitions are labeled with functions which
 - operate on inputs and memory,
 - update the memory and
 - produce outputs
 - All states are considered terminal.

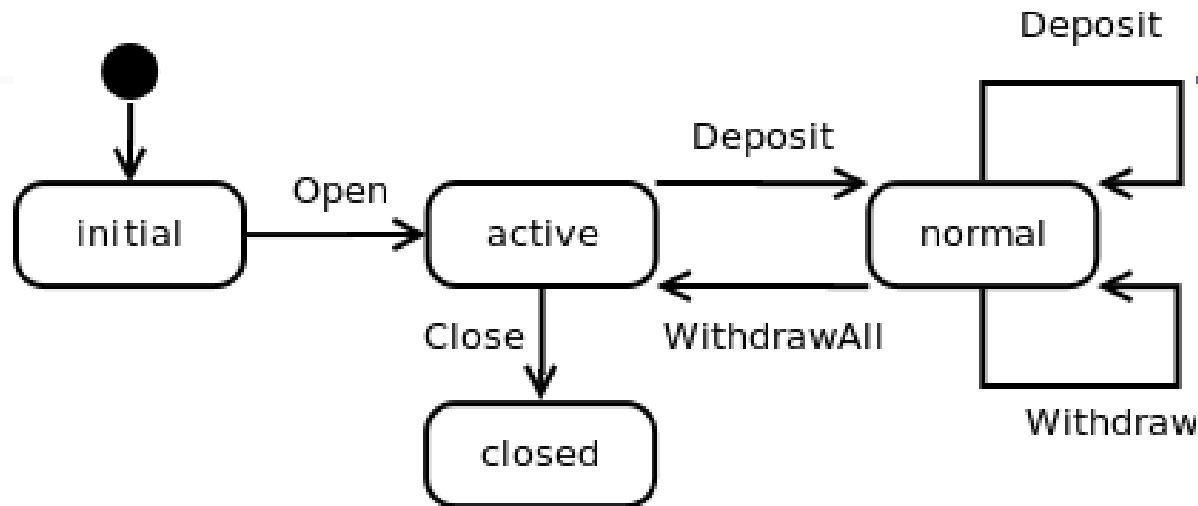
SXM definition

- An SXM consists of:
 - a set of states Q
 - a set of inputs Σ
 - a set of outputs Γ
 - a set M called memory
 - an initial state q_0
 - an initial memory m_0
 - a set Φ of processing functions, where each φ in Φ :
 - $\varphi : \Sigma \times M \rightarrow \Gamma \times M$
 - a function $F: Q \times \Phi \rightarrow Q$ called the transition function.

Processing functions

- A processing function consists of:
 - A guard condition depending on the input and the memory
 - An effect which
 - changes the memory and
 - produces an output

SXM Account



$\text{Open}(b, \text{open}()) = (\text{openOut}, 0)$

$\text{Deposit}(b, \text{deposit}(a)) = (\text{depositOut}, b + a)$

$\text{Withdraw}(b, \text{withdraw}(a)) = (\text{withdrawOut}, b - a)$ if $a < b$

$\text{WithdrawAll}(b, \text{withdraw}(a)) = (\text{withdrawOut}, 0)$ if $a = b$

$\text{Close}(b, \text{close}()) = (\text{closeOut}, b)$

Initial Memory
 $b = 0$

Streams

- Streams are sequences of symbols
- A SXM Z processes an input sequence and produces an output sequence, i.e. it computes a function:
 - $f_Z : \Sigma^* \rightarrow \Gamma^*$
- In each computation step:
 - An input is read from the input stream
 - The input triggers the firing of a processing function
 - An output is written on the output stream



FSM based Testing

Testing problem

The specification/model is a FSM.

The Implementation under Test (IUT) can be itself modelled by an (erroneous) FSM

We need to find as many faults in the IUT as possible

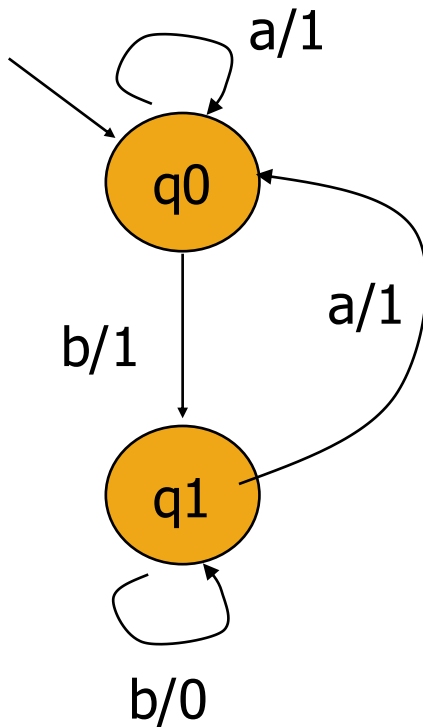
Ideally: the IUT that has passed all tests is fault-free

Faults in implementation

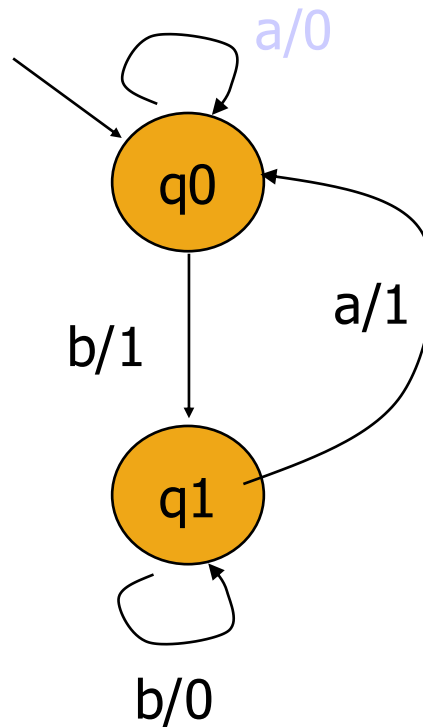
An FSM serves to specify the correct requirement or design of an application. Hence tests generated from an FSM target faults related to the FSM itself.

What faults are targeted by the tests generated using an FSM?

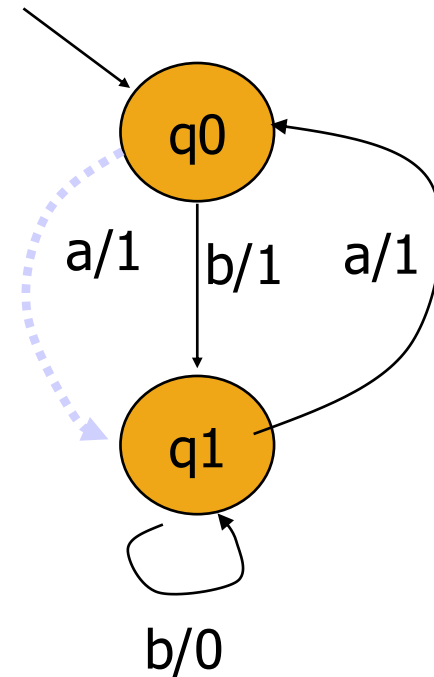
Fault model



Correct design

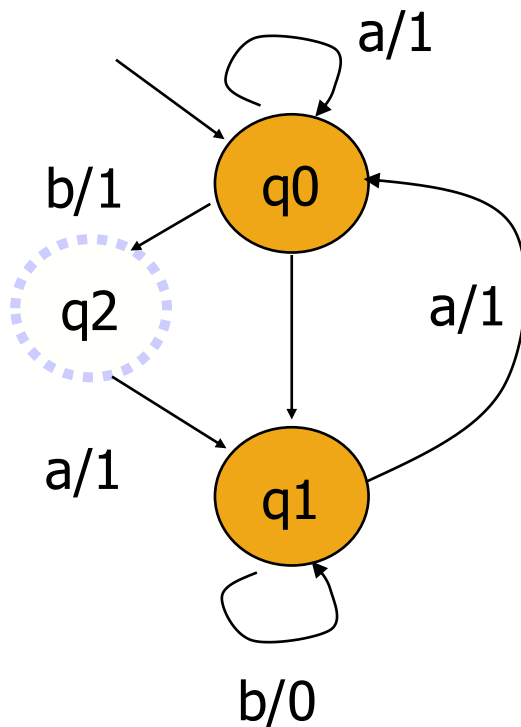


Operation error

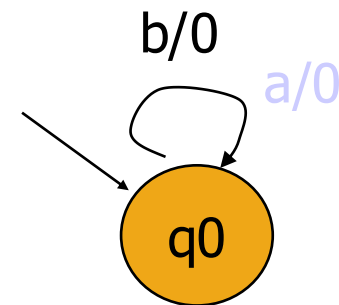


Transfer error

Fault model (contd.)



Extra state error



Missing state error

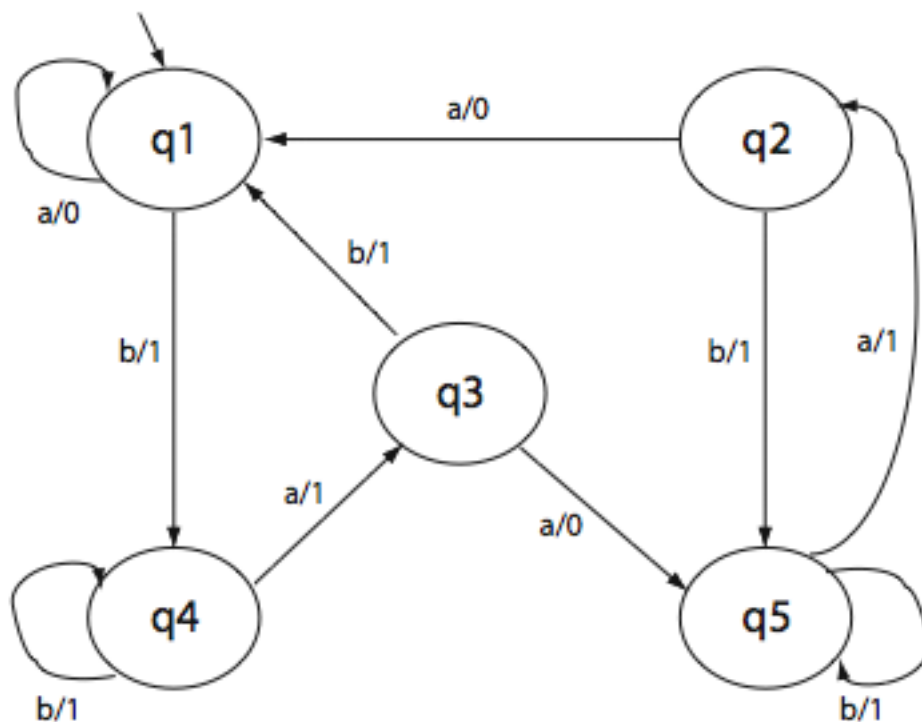
Test selection strategy based on a fault model

Define a fault model.

For each fault in de fault model, find a test that reveals the error.

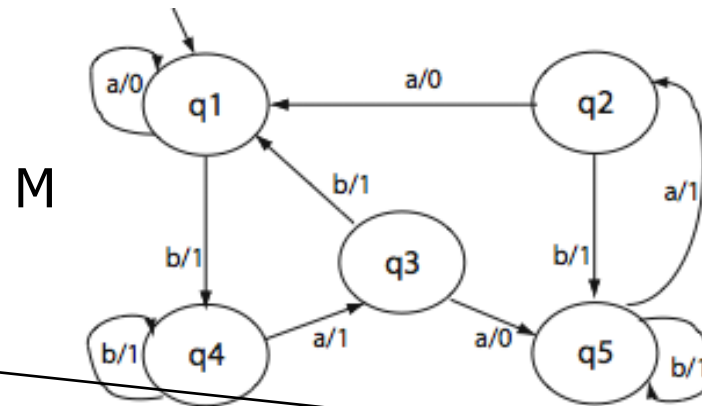
The test suite will comprise all such tests.

Example



Example 1: Testing an erroneous application

Correct design



$M(t1)=1101000$

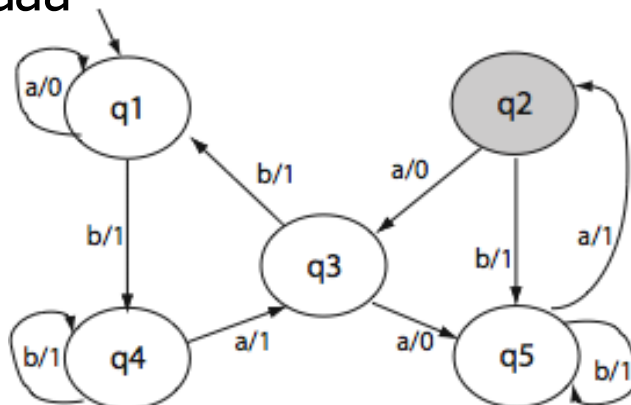
$M(t2)=11011$

Error revealing
test cases

$t1=baaaaaa$

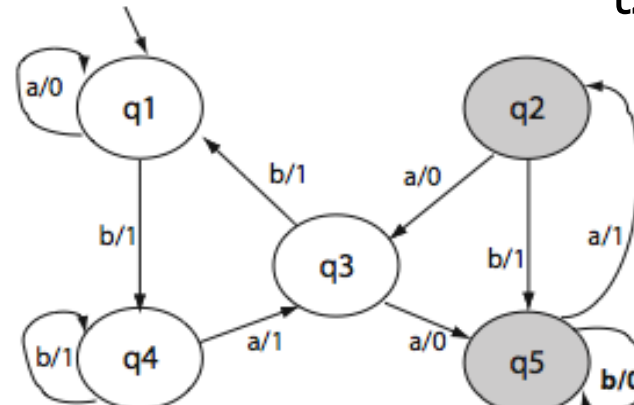
(a) Specification

$t2=baaba$



$M1(t1)=1101001$

(b) Transfer error in state q2.



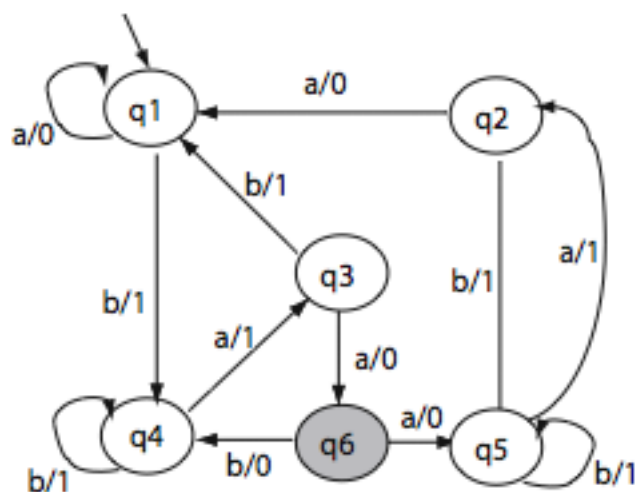
(c) Transfer error in state q2 and
operation error in state q5.

$M2(t2)=11001$

M1

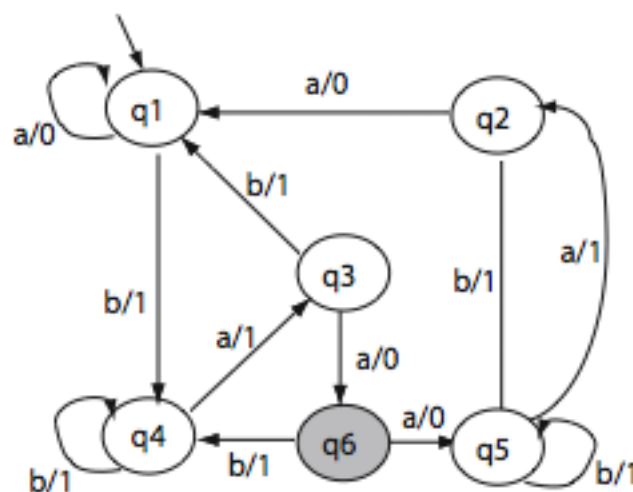
M2

Example 2: Extra state. $N=5$, $m=6$.



(a)

M1



(b)

M2

$t1=baaba$ $M(t1)=11011$ $M1(t1)=11001$

$t2=baaa$ $M(t2)=1101$ $M2(t2)=1100$

Coverage strategies

State cover – all states of the FMS model are reached

Transition cover – all transition are triggered “2-switch coverage”

n-switch coverage - tests combinations of $n+1$ consecutive transitions

Limitation: How is coverage related to fault detection?

State-detection strategies - DS

In a finite state machine (FSM), a **DS (distinguishing sequence)** is an input sequence that lets you tell states apart by observing the outputs.

Intuition: If you start the FSM in two different states and feed it the same input sequence, you'll watch the output sequence it produces. A distinguishing sequence is one that makes the output sequences different, so you can conclude "we must have started in state A, not state B."

State-detection strategies - UIO

A UIO (Unique Input/Output sequence) is an input sequence that uniquely identifies one particular state of a deterministic FSM by the output sequence it produces.

So: run the same input sequence x from any possible starting state; only state S produces that specific output sequence.

DS: one sequence separates every pair of states (identifies any unknown start state).

UIO: a sequence that separates one chosen state S from all the others (identifies that state).

A DS implies every state has a UIO (take the DS), but UIOs can exist even when no single global DS exists.

State-detection strategies - limitations

Both a DS and an UIO may not exist

Complex algorithms for finding a DS or an UIO in the worst-case scenario.

How state-detection is related to fault detection?

W-method for test generation (Chow)

Construct a test suite that reveals **any fault** in the Implementation under Test

W-method assumptions

Minimality: An FSM M is considered minimal if the number of states in M is less than or equal to any other FSM equivalent to M .

Completely specified: An FSM M is said to be completely specified if from each state in M there exists a transition for each input symbol.



The
University
Of
Sheffield.



CITY College
An International
Faculty Of
The University.

Overall algorithm used in the W-method

Overall algorithm used in the W-method

Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .

Overall algorithm used in the W-method

Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .

Step 2: Construct the characterization set W for M .

Overall algorithm used in the W-method

Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .

Step 2: Construct the characterization set W for M .

Step 3: (a) Construct the testing tree for M and (b) generate the transition cover set P from the testing tree.

Overall algorithm used in the W-method

Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .

Step 2: Construct the characterization set W for M .

Step 3: (a) Construct the testing tree for M and (b) generate the transition cover set P from the testing tree.

Step 4: Construct set Z from W and m .

Overall algorithm used in the W-method

Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .

Step 2: Construct the characterization set W for M .

Step 3: (a) Construct the testing tree for M and (b) generate the transition cover set P from the testing tree.

Step 4: Construct set Z from W and m .

Step 5: Desired test set = $P.Z$

Step 1: Estimation of m

This is based on a knowledge of the implementation.

Essential for the usability and precision of the method.

A too large m will yield very large test suites.

A too small m will affect the fault detection of the method.

Ideally $m - |Q|$ very small.

Step 2: Construction of W . What is W ?

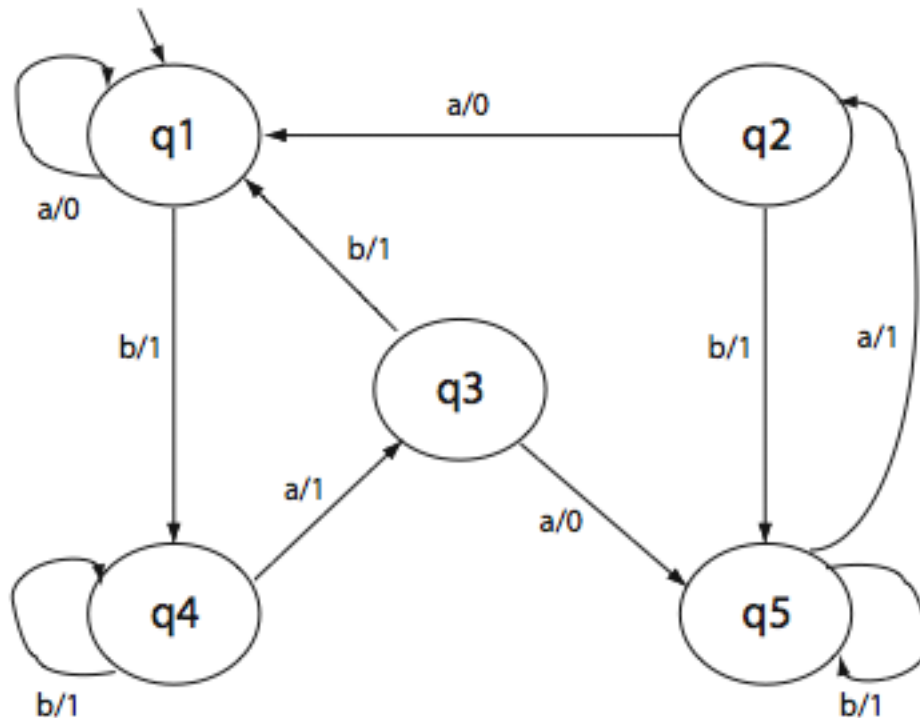
Let $M=(X, Y, Q, q_1, \delta, O)$ be a minimal and complete FSM.

W is a finite set of input sequences that distinguish the behavior of any pair of states in M . Each input sequence in W is of finite length.

Given states q_i and q_j in Q , W contains a string s such that:

$$O(q_i, s) \neq O(q_j, s)$$

Example of W



$W = \{baaa, aa, aaa\}$

$O(baaa, q1) = 1101$

$O(baaa, q2) = 1100$

Thus **baaa** distinguishes state $q1$ from $q2$ as $O(baaa, q1) \neq O(baaa, q2)$

Steps in the construction of W

Step 1: Construct a sequence of k -equivalence partitions of Q denoted as $P_1, P_2, \dots, P_m, m > 0$.

Step 2: Traverse the k -equivalence partitions in reverse order to obtain distinguishing sequence for each pair of states.

What is a k-equivalence partition of Q?

A k-equivalence partition of Q, denoted as P_k , is a collection of n finite sets $\Sigma_{k1}, \Sigma_{k2} \dots \Sigma_{kn}$ such that

$$\cup_{i=1}^n \Sigma_{ki} = Q$$

States in Σ_{ki} are k-equivalent.

If state u is in Σ_{ki} and v in Σ_{kj} for $i \neq j$, then u and v are k-distinguishable.

How to construct a k-equivalence partition?

Given an FSM M, construct a 1-equivalence partition, start with a tabular representation of M.

Current state	Output		Next state	
	a	b	a	b
q1	0	1	q1	q4
q2	0	1	q1	q5
q3	0	1	q5	q1
q4	1	1	q3	q4
q5	1	1	q2	q5

Construct 1-equivalence partition

Group states identical in their Output entries. This gives us 1-partition P_1 consisting of $\Sigma_1 = \{q1, q2, q3\}$ and $\Sigma_2 = \{q4, q5\}$.

Σ	Current state	Output		Next state	
		a	b	a	b
1	q1	0	1	q1	q4
	q2	0	1	q1	q5
	q3	0	1	q5	q1
2	q4	1	1	q3	q4
	q5	1	1	q2	q5

Construct 2-equivalence partition: Rewrite P_1 table

Rewrite P_1 table. Remove the output columns. Replace a state entry q_i by q_{ij} where j is the group number in which lies state q_i .

Σ	Current state	Next state	
		a	b
1	q_1	q_{11}	q_{42}
	q_2	q_{11}	q_{52}
	q_3	q_{52}	q_{11}
2	q_4	q_{31}	q_{42}
	q_5	q_{21}	q_{52}

P_1 Table

Group number

Construct 2-equivalence partition: Construct P_2 table

Group all entries with identical second subscripts under the next state column. This gives us the P_2 table. Note the change in second subscripts.

P_2 Table

Σ	Current state	Next state	
		a	b
1	q1	q11	q43
	q2	q11	q53
2	q3	q53	q11
3	q4	q32	q43
	q5	q21	q53

Construct 3-equivalence partition: Construct P_3 table

Group all entries with identical second subscripts under the next state column. This gives us the P_3 table. Note the change in second subscripts.

P_3 Table

Σ	Current state	Next state	
		a	b
1	q1	q11	q43
	q2	q11	q54
2	q3	q54	q11
3	q4	q32	q43
4	q5	q21	q54

Construct 4-equivalence partition: Construct P_4 table

Continuing with regrouping and relabeling, we finally arrive at P_4 table.

P_4 Table

Σ	Current state	Next state	
		a	b
1	q1	q11	q44
2	q2	q11	q55
3	q3	q55	q11
4	q4	q33	q44
5	q5	q22	q55

k-equivalence partition: Convergence of the process

The process is guaranteed to converge.

When the process converges, and the machine is minimal, each state will be in a separate group.

The next step is to obtain the distinguishing strings for each state.

Finding the distinguishing sequences: Example

Let us find a distinguishing sequence for states q_1 and q_2 .

Find tables P_i and P_{i+1} such that (q_1, q_2) are in the same group in P_i and different groups in P_{i+1} . We get P_3 and P_4 .

Initialize $z = \varepsilon$. Find the input symbol that distinguishes q_1 and q_2 in table P_3 . This symbol is b . We update z to $z.b$. Hence z now becomes b .

Finding the distinguishing sequences: Example (contd.)

The next states for q_1 and q_2 on b are, respectively, q_4 and q_5 .

We move to the P_2 table and find the input symbol that distinguishes q_4 and q_5 . Let us select a as the distinguishing symbol. Update z which now becomes ba .

The next states for states q_4 and q_5 on symbol a are, respectively, q_3 and q_2 . These two states are distinguished in P_1 by a and b . Let us select a . We update z to baa .

Finding the distinguishing sequences: Example (contd.)

The next states for q_3 and q_2 on a are, respectively, q_1 and q_5 .

Moving to the original state transition table we obtain a as the distinguishing symbol for q_1 and q_5

We update z to $baaa$. This is the farthest we can go backwards through the various tables. $baaa$ is the desired distinguishing sequence for states q_1 and q_2 . Check that $o(q_1,baaa) \neq o(q_2,baaa)$.

Finding the distinguishing sequences: Example (contd.)

Using the procedure analogous to the one used for q_1 and q_2 , we can find the distinguishing sequence for each pair of states. This leads us to the following characterization set for our FSM.

$$W = \{a, aa, aaa, baaa\}$$

Chow's method: where are we?

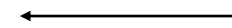
Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .



Done

Step 2: Construct the characterization set W for M .

Step 3: (a) Construct the testing tree for M and (b) generate the transition cover set P from the testing tree.



Next (a)

Step 4: Construct set Z from W and m .

Step 5: Desired test set= $P.Z$

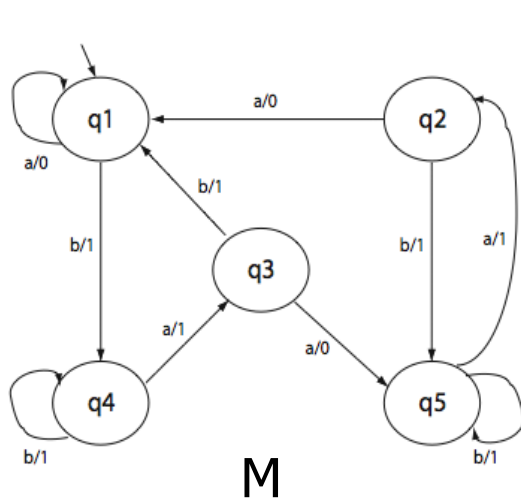
Step 3: (a) Construct the testing tree for M

A testing tree of an FSM is a tree rooted at the initial state. It contains at least one path from the initial state to the remaining states in the FSM. Here is how we construct the testing tree.

State q_0 , the initial state, is the root of the testing tree. Suppose that the testing tree has been constructed until level k . The $(k+1)$ th level is built as follows.

Select a node n at level k . If n appears at any level from 1 through k , then n is a leaf node and is not expanded any further. If n is not a leaf node then we expand it by adding a branch from node n to a new node m if $\delta(n, x) = m$ for $x \in X$. This branch is labeled as x . This step is repeated for all nodes at level k .

Example: Construct the **testing tree** for M



Level 1

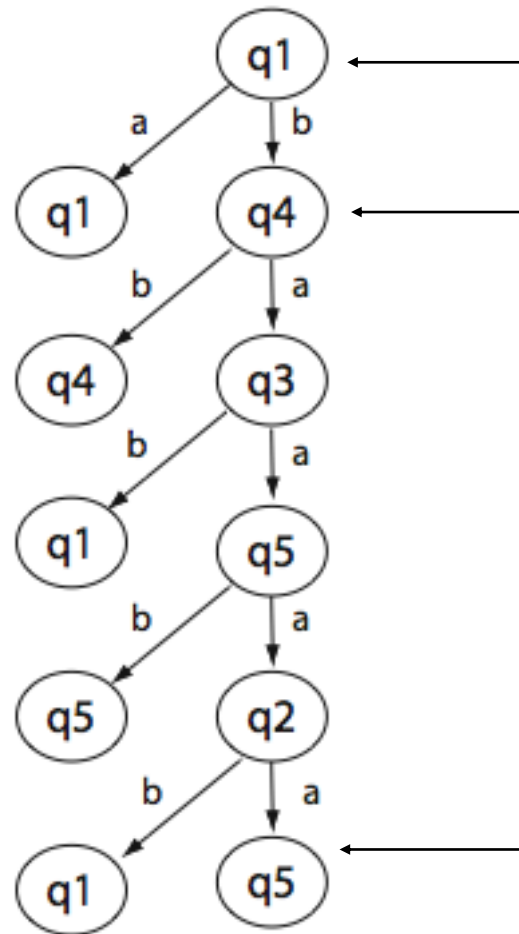
Level 2

Level 3

Level 4

Level 5

Level 6



Start here, initial state is the root.

q1 becomes leaf, q4 can be expanded.

•

•

•

No further expansion possible

Chow's method: where are we?

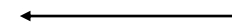
Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .



Done

Step 2: Construct the characterization set W for M .

Step 3: (a) Construct the testing tree for M and (b) generate the transition cover set P from the testing tree.



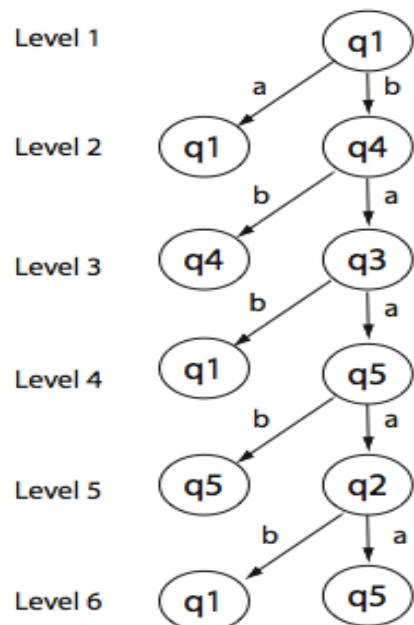
Next, (b)

Step 4: Construct set Z from W and m .

Step 5: Desired test set= $P.Z$

Step 3: (b) Find the transition cover set from the testing tree

A **transition cover set** P is a set of all strings representing sub-paths, starting at the root, in the testing tree. Concatenation of the labels along the edges of a sub-path is a string that belongs to P . The empty string (ϵ) also belongs to P .



$P = \{\epsilon, a, b, bb, ba, bab, baa, baab, baaa, baaab, baaaa\}$

Chow's method: where are we?

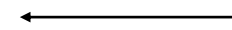
Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .



Done

Step 2: Construct the characterization set W for M .

Step 3: (a) Construct the testing tree for M and (b) generate the transition cover set P from the testing tree.



Done

Step 4: Construct set Z from W and m .



Next

Step 5: Desired test set= $P.Z$

Step 4: Construct set Z from W and m

Given that X is the input alphabet and W the characterization set, we have:

$$Z = X^0.W \cup X^1.W \cup \dots X^{m-1-n}.W \cup X^{m-n}.W$$

For $m=n$, we get

$$Z = X^0.W = W$$

For $X=\{a, b\}$, $W=\{a, aa, aaa, baaa\}$, $m=6$

$$\begin{aligned} Z &= W \cup X^1.W = \{a, aa, aaa, baaa\} \cup \{a, b\}.\{a, aa, aaa, baaa\} \\ &= \{a, aa, aaa, baaa, aa, aaa, aaaa, baaaa, ba, baa, baaa, \\ &\quad bbaaaa\} \end{aligned}$$

Chow's method: where are we?

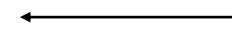
Step 1: Estimate the maximum number of states (m) in the correct implementation of the given FSM M .



Done

Step 2: Construct the characterization set W for M .

*Step 3: (a) Construct the **testing tree** for M and (b) generate the transition cover set P from the testing tree.*



Done

Step 4: Construct set Z from W and m .



Done

Step 5: Desired **test set**= $P.Z$



Next

Step 5: Desired test set=P.Z

The test inputs based on the given FSM M can now be derived as:

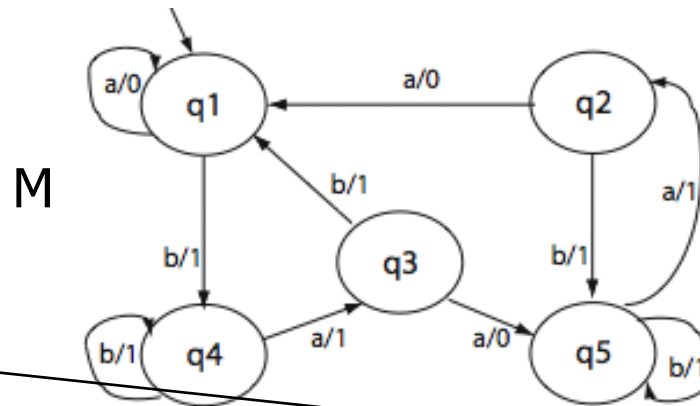
$$T=P.Z$$

Do the following to test the implementation:

1. Find the expected response to each element of T.
2. Generate test cases for the application. Note that even though the application is modeled by M, there might be variables to be set before it can be exercised with elements of T.
3. Execute the application and check if the response matches. Reset the application to the initial state after each test.

Example 1: Testing an erroneous application

Correct design



$M(t1)=1101000$

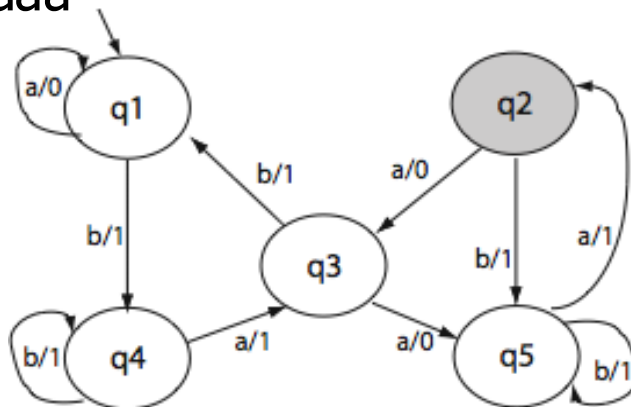
$M(t2)=11011$

Error revealing
test cases

$t1=baaaaaa$

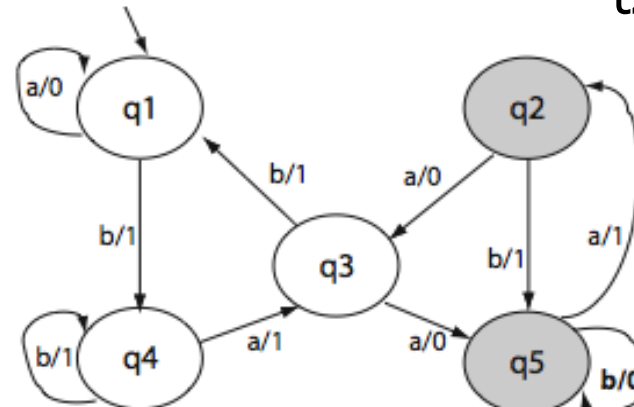
(a) Specification

$t2=baaba$



$M1(t1)=1101001$

(b) Transfer error in state q2.



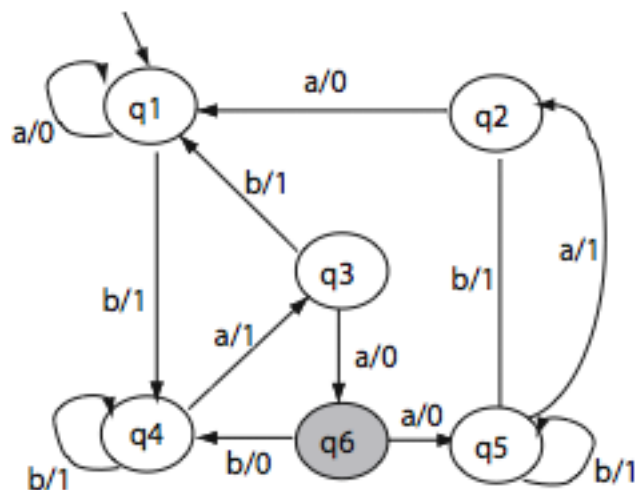
(c) Transfer error in state q2 and
operation error in state q5.

$M2(t2)=11001$

M1

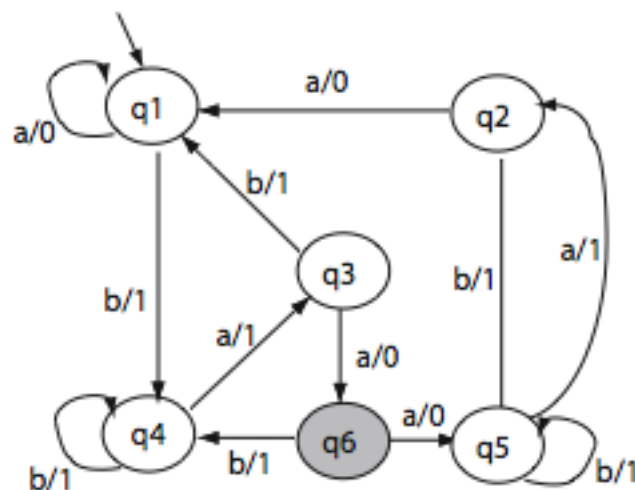
M2

Example 2: Extra state. $N=5$, $m=6$.



(a)

M1



(b)

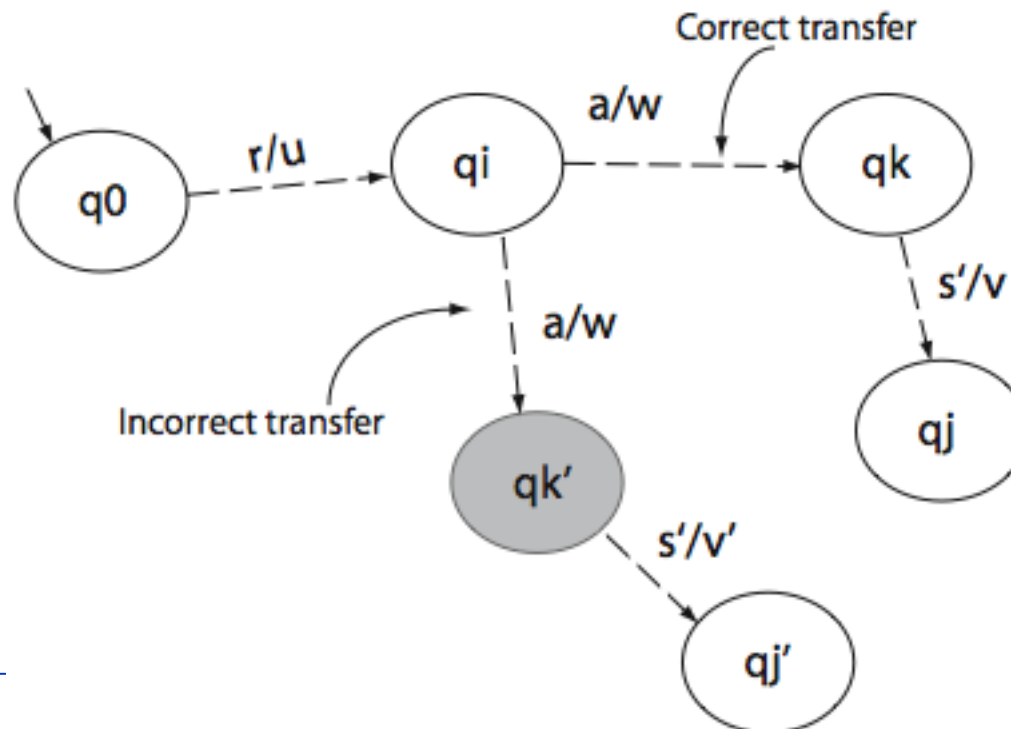
M2

$t1=baaba$ $M(t1)=11011$ $M1(t1)=11001$

$t2=baaa$ $M(t2)=1101$ $M2(t2)=1100$

Error detection process: in-class discussion

Given $m=n$, each test case t is of the form $r.s$ where r is in P and s in W . r moves the application from initial state q_0 to state q_i . Then, $s=as'$ takes it from q_i to state q_j or q_j' .



The partial W (Wp) method

Tests are generated from minimal, complete, and connected FSM.

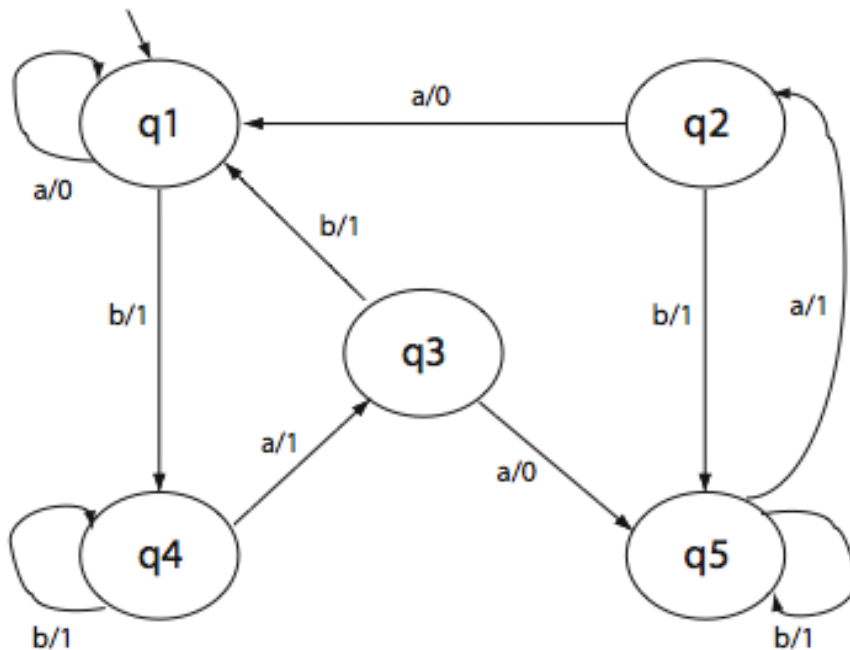
Size of tests generated is generally smaller than that generated using the W-method.

Test generation process is divided into two phases: **Phase 1**: Generate a test set using the state cover set (S) and the characterization set (W).
Phase 2: Generate additional tests using a subset of the transition cover set and state identification sets.

What is a state cover set? A state identification set?

State cover set

Given FSM M with input alphabet X , a state cover set S is a finite non-empty set of strings over X^* such that for each state q_i in Q , there is a string in S that takes M from its initial state to q_i .



$$S = \{\epsilon, b, ba, baa, baaa\}$$

S is always a subset of the transition cover set P . Also, S is not necessarily unique.

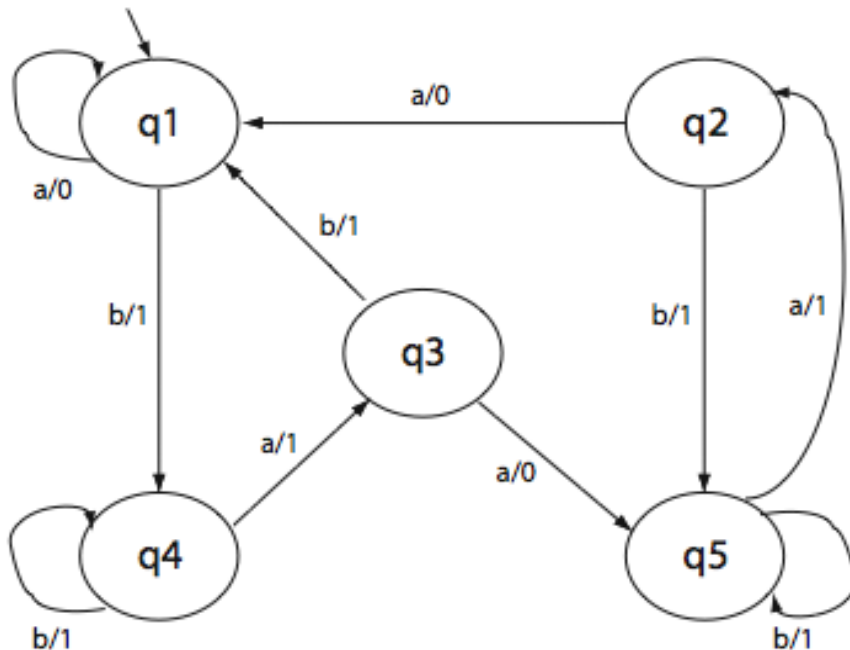
State identification set

Given an FSM M with Q as the set of states, an identification set for state $q_i \in Q$ is denoted by W_i and has the following properties:

- (a) $W_i \subseteq W$, $1 \leq i \leq n$ [Identification set is a subset of W .]
- (b) $O(q_i, s) \neq O(q_j, s)$, for $1 \leq j \leq n$, $j \neq i$, $s \in W_i$ [For each state other than q_i , there is a string in W_i that distinguishes q_i from q_j .]
- (c) No subset of W_i satisfies property (b). [W_i is minimal.]

State identification set: Example

Last element of the output string



$$W_1 = W_2 = \{baaa, aa, a\}$$

$$W_3 = \{a\} \quad W_4 = W_5 = \{a, aaa\}$$

Si	Sj	X	o(Si,x)	o(Sj,x)
1	2	baaa	1	0
	3	aa	0	1
	4	a	0	1
	5	a	0	1
2	3	aa	0	1
	4	a	0	1
	5	a	0	1
3	4	a	0	1
	5	a	0	1
4	5	aaa	1	0

Wp method: Example: Step 1: Compute S , P , W , W_i , $\mathcal{W}$

$S = \{\epsilon, b, ba, baa, baaa\}$

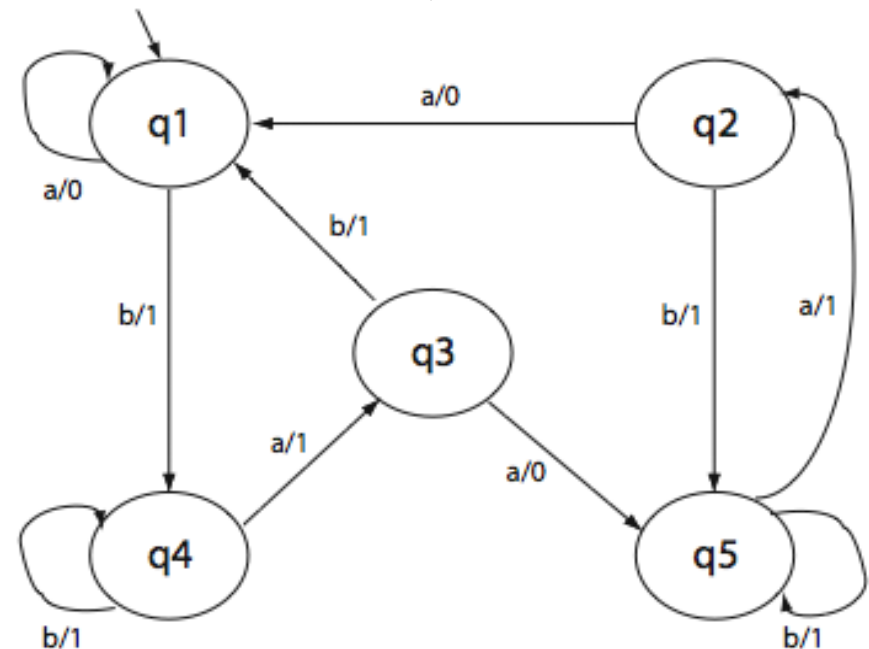
$P = \{\epsilon, a, b, bb, ba, bab, baa, baab, baaa, baaab, baaaa\}$

$W_1 = W_2 = \{baaa, aa, a\}$

$W_3 = \{a\}$ $W_4 = W_5 = \{a, aaa\}$

$W = \{a, aa, aaa, baaa\}$

$\mathcal{W} = \{W_1, W_2, W_3, W_4, W_5\}$



Wp method: Example: Step 2: Compute T1 [m=n]

$T1 = S. W = \{\epsilon, b, ba, baa, baaa\} \cdot \{a, aa, aaa, baaa\}$

Elements of T1 ensure that the each state of the FSM is covered and distinguished from the remaining states.

Wp method: Example: Step 3: Compute R and δ [m=n]

$$\begin{aligned} R = P - S &= \{\epsilon, a, b, bb, ba, bab, baa, baab, baaa, baaab, baaaa\} - \{\epsilon, b, \\ &\quad ba, baa, baaa\} \\ &= \{a, bb, bab, baab, baaab, baaaa\} \end{aligned}$$

Let each element of R be denoted as $r_{i1}, r_{i2}, \dots, r_{ik}$.

$\delta(r_{ik}, m) = q_{ij}$, where $m \in X$ (the alphabet)

Wp method: Example: Step 4: Compute T2 [m=n]

$T2 = R \otimes W = \bigcup_{(j=1)}^k (r_{ij} \cdot W_{ij})$, where W_{ij} is the identification set for state q_{ij} .

$$\delta(q1, a) = q1 \qquad \delta(q1, bb) = q4 \qquad \delta(q1, bab) = q5$$

$$\delta(q1, baab) = q5 \qquad \delta(q1, baaab) = q5 \qquad \delta(q1, baaaa) = q1$$

$$\begin{aligned} T2 &= (\{a\} \cdot W_1) \cup (\{bb\} \cdot W_4) \cup (\{bab\} \cdot W_5) \cup (\{baab\} \cdot W_5) \cup \\ &\quad \{baaab\} \cdot W_5) \cup (\{baaaa\} \cdot W_1) \\ &= \{abaaa, aaa, aa\} \cup \{bba, bbaaa\} \cup \{baba, babaaa\} \cup \\ &\quad \{baaba, baabaaa\} \cup \{baaaba, baaabaaa\} \cup \{baaaabaaa, \\ &\quad baaaaaa, baaaaa\} \end{aligned}$$

Wp method: Example: Savings

Test set size using the W method= 44

Test set size using the Wp method= 34 (20 from T1+14 from T2)

Testing using the Wp method

Testing proceeds in **two phases**.

Tests from T1 are applied in **phase 1**. Tests from T2 are applied in **phase 2**.

While tests from phase 1 ensure state coverage, they do not ensure all transition coverage. Also, even when tests from phase cover all transitions, they do not apply the state identification sets and hence not all transfer errors are guaranteed to be revealed by these tests.

Wp method:

Both sets T1 and T2 are computed a bit differently, as follows:

$T1 = S. X[m-n]$, where $X[m-n]$ is the set union of X^i , $1 \leq i \leq (m-n)$

$T2 = T2 = R. X[m-n] \otimes \mathcal{W}$

Summary

Behavior of a large variety of applications can be modeled using finite state machines (FSM). GUIs can also be modeled using FSMs

The W and the Wp methods are automata theoretic methods to generate tests from a given FSM model.

Tests so generated are guaranteed to detect all operation errors, transfer errors, and missing/extra state errors in the implementation given that the FSM representing the implementation is complete, connected, and minimal. *What happens if it is not?*

Summary (contd.)

Automata theoretic techniques generate tests superior in their fault detection ability than their control-theoretic counterparts.

Control-theoretic techniques, that are often described in books on software testing, include branch cover, state cover, boundary-interior, and n-switch cover.

The size of tests sets generated by the W method is larger than generated by the Wp method while their fault detection effectiveness are the same.



SXM Testing method

SXM Testing method

- SXMs offer a testing method that ensures the conformance of an implementation under test (IUT) to a specification.
- The production of a finite test set is based on a generalization of the W-method.
- This generalization is possible only if some “design-for-test” conditions are met.

Design-for-test conditions

- Taken from control theory
- **Controllability**
 - The ability to move a system in its state space by admissible inputs
- **Observability**
 - The ability to determine the system state (or behaviour) by using the inputs and the outputs

Testing problem

The specification/model is a SXM.

The Implementation under Test (IUT) can be itself modelled by an (erroneous) SXM

Additional requirement: two SXMs have the same processing functions (these are tested separately – reductionist approach)

W-method

- Applied to the associated finite automaton
 $A = (\Phi, Q, F, q_0)$
- When the test set is applied to two automata with the same input alphabet it can determine whether the automata are equivalent.
- When applied to SXMs the sequences of processing functions need to be converted to sequences of inputs.
- If two SXMs have the same processing functions, then by applying the W-method we can determine the functional equivalence of the 2 models.

Mismatches

- The SXM testing method can discover:
 - ☐ Missing states
 - ☐ Extra states
 - ☐ Missing transitions
 - ☐ Extra transitions

Variants of the method

Design-for-test conditions: controllability and observability

Controllability may sometimes be problematic to enforce

The controllability condition can be relaxed at the expense of a more complex generation algorithm and of a larger test suite



JSXM

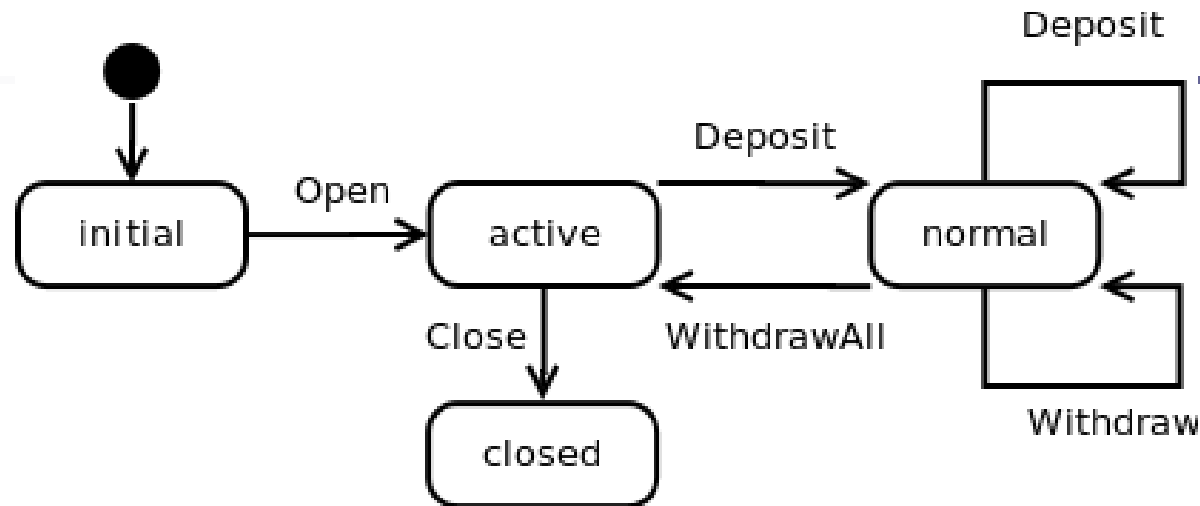
A tool for SXMs

<http://www.jsxm.org>

JSXM

- JSXM is a model-based testing tool.
- The tool is implemented in Java and allows the automated generation of test cases based on a model of the software to be tested.
- The model is expressed as a Stream X-machine (SXM). With the help of JSXM the specified model can be used for:
 - ☐ Model Animation
 - ☐ Test Generation
 - ☐ Test Transformation

SXM Account



$\text{Open}(b, \text{open}()) = (\text{openOut}, 0)$

$\text{Deposit}(b, \text{deposit}(a)) = (\text{depositOut}, b + a)$

$\text{Withdraw}(b, \text{withdraw}(a)) = (\text{withdrawOut}, b - a)$ if $a < b$

$\text{WithdrawAll}(b, \text{withdraw}(a)) = (\text{withdrawOut}, 0)$ if $a = b$

$\text{Close}(b, \text{close}()) = (\text{closeOut}, b)$

Initial Memory
 $b = 0$

SXM Specification in XML:

Specification top structure

```
<SXM name="Account">  
  <states>  
    <initialState>  
    <transitions>  
  </states>  
  <memory>  
  <inputs>  
  <outputs>  
  <functions>  
</SXM>
```

SXM Specification in XML:

States

```
<states>  
  <state name="initial" />  
  <state name="opened" />  
  <state name="closed" />  
  <state name="normal" />  
</states>  
  
<initialState state="initial" />
```


SXM Specification in XML:

Transitions labelled with functions

```
<transitions>
<transition from="initial" function="openF" to="opened" />
<transition from="opened" function="closeF" to="closed" />
<transition from="opened" function="depositF" to="normal" />
<transition from="normal" function="depositF" to="normal" />
<transition from="normal" function="withdrawN" to="normal" />
<transition from="normal" function="withdraw0" to="opened" />
</transitions>
```

SXM Specification in XML:

Inputs

```
<inputs>
<input name="open" />
<input name="close" />
<input name="deposit">
    <arg name="amount" type="xs:int" />
</input>
<input name="withdraw">
    <arg name="amount" type="xs:int" />
</input>
</inputs>
```

SXM Specification in XML:

Outputs

```
<outputs>
  <output name="openOut" />
  <output name="closeOut" />
  <output name="depositOut">
    <result name="amount" type="xs:int" />
  </output>
  <output name="withdrawOut">
    <result name="amount" type="xs:int" />
  </output>
</outputs>
```

SXM Specification in XML: Memory (Java inline code)

```
<memory>  
<declaration>  
    int balance;  
</declaration>  
<initial>  
    balance = 0;  
</initial>  
</memory>
```

SXM Specification in XML: Functions (Java inline code)

```
<functions>
```

```
...
```

```
<function name="depositF" input="deposit" output="depositOut">
```

```
  <precondition>
```

```
    deposit.get_amount > 0;
```

```
  </precondition>
```

```
  <effect>
```

```
    balance = balance + deposit.get_amount;
```

```
    depositOut.amount = deposit.get_amount;
```

```
  </effect>
```

```
</function>
```

```
...
```

```
</functions>
```

Tools

- **Parser and Syntax Validator**
 - Parses and validates XML specifications
- **Animator**
 - Reads sequences of inputs in XML
 - Outputs results on screen and in an XML file
 - **GUI**: graphically animates the state diagram
- **Test-generator**
 - Generates an XML file with all test cases
- **Test transformers**
 - Transform XML test cases to specific testing framework (i.e. JUnit for Java implementations)

Test Transformation

- The test cases generated by the JSXM test generation are in XML (i.e. language independent).
- A transformation process is necessary to transform the **abstract test cases** to **concrete test cases**.
- The JUnit transformer:
 - abstract test cases → JUnit test cases
- The generated JUnit java class intends to test a Java adapter class.
 - The adapter wraps all method calls of the class under test so that inputs and outputs are compatible to those of the specification.



SXM Formalism

Observability

- SXM testing strategy requires output-distinguishability.
- A set Φ of functions (type of the machine) is **output-distinguishable** if:
 - $\forall \varphi_1, \varphi_2 \in \Phi$, **if** $\exists m, m_1, m_2 \in M, \sigma \in \Sigma, \gamma \in \Gamma$ such that:
 $\varphi_1(\mathbf{m}, \sigma) = (\gamma, m_1)$ and $\varphi_2(\mathbf{m}, \sigma) = (\gamma, m_2)$
then $\varphi_1 = \varphi_2$
- If two functions are applicable at the same memory value and the same input they should produce different outputs.
- **Output-distinguishability** is a condition which is easily satisfied.
 - SXM Specifications are usually output-distinguishable without any extra effort.
 - Implementations can be made output-distinguishable by adding appropriate response messages.
 - A better approach (used by JSXM) is to specify test adapters for that purpose.

Controllability

- Input-completeness. A set Φ of functions is **input-complete** if:
 - For every function φ :
 - $\forall m, \exists \sigma$, that $(m, \sigma) \in \text{dom}(\varphi)$
 - Any function can be triggered at any memory value with some input
- A SXM which is not input-complete can be modified to an input-complete SXM by
 - Adding **extra inputs** for the undefined memory values
 - Adding **extra outputs** for output-distinguishability.
- **Input-completeness** was a necessary condition in SXM testing strategy for:
 - The state cover set
 - The characterization set
 - The fundamental test function
- Later updates of the testing method have relaxed this requirement.

W-method

- Applied on the associated finite automaton
 $A = (\Phi, Q, F, q_0)$
- **State cover S:**
 - A set of sequences such that all states are reachable from the initial state.
- **Characterization set W:**
 - a set of sequences for which any two distinct states of the automaton are distinguishable.

W-method Test Set

- The W-method is used to generate the Test Set Y_k of the associated automaton.

$$Y_k = S(\{\varepsilon\} \cup \Phi^1 \cup \Phi^2 \cup \dots \cup \Phi^{k+1})W$$

- A test set consists of sequences of processing functions that are able to:
 - Reach every state by applying the **state cover functions**;
 - at that state apply all **tuples of functions** of maximum length $k+1$;
 - and distinguish the resulting state by applying the **characterisation set functions**.

The test function

- The **goal** of the test function **t** is to transform any function sequence **u** to an input sequence which exercises the **longest admissible prefix of u** and also exercises the function following this prefix.
- $t(\varphi_1, \dots, \varphi_n) = t(\varphi_1, \dots, \varphi_i) = \sigma_1, \dots, \sigma_i$
 - Case 1: $\varphi_1, \dots, \varphi_n$ is admissible, and $i = n$
 - Case 2: $\varphi_1, \dots, \varphi_{i-1}$ is admissible, and $\varphi_1, \dots, \varphi_{i-1}, \varphi_i$ is not.
 - $(m_0, \sigma_1, \dots, \sigma_i) \in \text{dom}(|\varphi_1 \dots \varphi_i|)$
- *Input-completeness guarantees that there is always an input sequence for every function sequence.*